

Lessa?! | لسه؟

Smart Government Queue Tracking Platform

Software Requirements Specification (SRS)

Version 1.0 — Hackathon MVP Edition | June 2026

Version 1.0 | June 2026 | Hackathon MVP Edition

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for Lessa?! version 1.0 (MVP). It serves as the authoritative reference for design, development, testing, and acceptance validation by all team members.

1.2 Scope

Lessa?! is a web-based queue management platform for a single government service branch. It enables citizens to remotely book, track, and manage queue slots; enables counter staff to call and manage tickets; enables supervisors to monitor operations; and provides public visitors with live congestion data.

1.3 Glossary

Term	Definition
Branch	A single government service location with one or more service windows
Service	A specific transaction type offered at a branch (e.g., "National ID Renewal")
Window	A physical counter staffed by one employee, serving one service at a time
Queue Slot	A capacity unit in the daily queue for a service type
Ticket	A citizen's specific booking record with a unique queue number
Ticket Number	Formatted queue identifier: {ServiceCode}-{DailySequence} e.g., NID-042
Check-in	Action confirming a citizen's physical presence at the branch
Grace Period	Time after a ticket is "called" before it auto-transitions to no_show
Predicted Wait	AI-estimated time until a citizen's ticket will be called
SSE	Server-Sent Events — unidirectional real-time push from server to browser
RBAC	Role-Based Access Control
Audit Log	Immutable record of all critical system actions
No-Show	Ticket state when citizen fails to respond/arrive within grace period
Public Display Board	Unauthenticated large-screen view showing current/next ticket numbers

2. System Overview

2.1 Product Perspective

Lessa?! is a standalone web application consisting of: a React SPA frontend, a Node.js/Express modular monolith backend (REST API), a PostgreSQL relational database, and an optional Redis cache for queue state and session management.

2.2 User Classes

User Class	Auth Required	Tech Proficiency	Interface
------------	---------------	------------------	-----------

Guest	No	Low	Public web (mobile + desktop)
Citizen	Yes (phone)	Low–Medium	Mobile-first web app
Counter Staff	Yes	Medium	Desktop-optimized web app
Branch Supervisor	Yes	Medium	Desktop web app
System Admin	Yes	High	Admin dashboard web app

2.3 Operating Environment

- Client: Modern web browser (Chrome 90+, Safari 14+, Firefox 88+, Edge 90+)
- Mobile: iOS Safari 14+, Android Chrome 90+; responsive touch-optimized layout
- Server: Node.js 20 LTS on Ubuntu 22.04+
- Database: PostgreSQL 15+ with Redis 7+ optional cache
- Hosting: Render, Railway, or single VPS; HTTPS required

3. Functional Requirements

FR-01: Authentication & Authorization

ID	Requirement	Priority
FR-01-01	System shall allow citizens to register with phone number and full name	Must
FR-01-02	System shall support phone-based login with OTP (mocked/simplified for MVP)	Must
FR-01-03	System shall issue JWT access tokens (15-min expiry) and refresh tokens (7-day)	Must
FR-01-04	System shall enforce RBAC for all protected endpoints	Must
FR-01-05	System shall allow admins to assign roles to users	Must
FR-01-06	System shall invalidate refresh tokens on logout	Must
FR-01-07	System shall allow guests to access public endpoints without authentication	Must
FR-01-08	System shall lock accounts after 5 consecutive failed login attempts for 30 minutes	Should
FR-01-09	System shall log all authentication events (login, logout, failed attempts)	Must

FR-02: User Management

ID	Requirement	Priority
FR-02-01	Admin shall be able to create, update, deactivate, and delete users	Must
FR-02-02	Admin shall be able to assign staff to windows	Must
FR-02-03	Citizens shall be able to update their own profile (name only; phone requires re-verification)	Should

FR-02-04	Admin shall be able to assign and revoke roles	Must
FR-02-05	Deactivated users shall be denied login	Must
FR-02-06	System shall prevent deletion of users with active tickets	Must

FR-03: Service Catalog

ID	Requirement	Priority
FR-03-01	Admin shall be able to create services with name (AR + EN), code, avg handling time, and daily capacity	Must
FR-03-02	Admin shall be able to associate services with one or more windows	Must
FR-03-03	Admin shall be able to deactivate a service (no new bookings; existing unaffected)	Must
FR-03-04	System shall expose service list publicly (for guests and booking flow)	Must
FR-03-05	Admin shall be able to set estimated average handling time per service	Must
FR-03-06	System shall display service load (current queue length / daily capacity) for public view	Must

FR-04: Booking Management

ID	Requirement	Priority
FR-04-01	Authenticated citizens shall be able to book a queue slot for any active service	Must
FR-04-02	System shall auto-assign a queue number on booking (format: {ServiceCode}-{DailySequence})	Must
FR-04-03	System shall enforce max one active booking per citizen per day	Must
FR-04-04	System shall enforce booking only during branch operating hours	Must
FR-04-05	System shall enforce daily capacity limits; reject bookings when full	Must
FR-04-06	Citizens shall be able to cancel a booking subject to cancellation cutoff rules	Must
FR-04-07	Citizens shall be able to change the service type of a booking (if no other active booking)	Should
FR-04-08	On cancellation, system shall free the slot and allow re-booking by other citizens	Must
FR-04-09	System shall send in-app notification on booking confirmation	Must
FR-04-10	System shall prevent booking by deactivated users	Must

FR-05: Queue Engine

ID	Requirement	Priority
FR-05-01	Queue engine shall maintain an ordered list of tickets per service, sorted by booking time	Must

FR-05-02	Queue engine shall atomically assign queue numbers to prevent duplicates	Must
FR-05-03	Staff shall be able to call the next ticket for their assigned window	Must
FR-05-04	When a ticket is called, system shall update ticket state to "called" and start grace period timer	Must
FR-05-05	If citizen does not check in within grace period, ticket shall auto-transition to no_show	Must
FR-05-06	Staff shall be able to manually mark a ticket as done, skipped, or no_show	Must
FR-05-07	Staff shall be able to re-call a previously skipped ticket (with audit log entry)	Should
FR-05-08	Queue engine shall recalculate positions for all waiting tickets after each state change	Must
FR-05-09	System shall support concurrent-safe queue operations (optimistic locking)	Must
FR-05-10	System shall expose current queue state for a service publicly (count, next number)	Must

FR-06: Ticket Tracking

ID	Requirement	Priority
FR-06-01	Citizens shall be able to view their current ticket status and position in queue	Must
FR-06-02	System shall provide real-time or near-real-time queue position updates (SSE or polling ≤30s)	Must
FR-06-03	System shall display predicted wait time on the citizen's ticket view	Must
FR-06-04	System shall display estimated time of service on the citizen's ticket view	Should
FR-06-05	Citizens shall be able to perform manual check-in from their ticket view	Must
FR-06-06	Citizens shall receive an in-app notification when their ticket is "called"	Must
FR-06-07	System shall show ticket status history to the citizen (timeline view)	Could

FR-07: Window Operations

ID	Requirement	Priority
FR-07-01	Supervisors shall be able to open and close windows	Must
FR-07-02	Supervisors shall be able to assign staff members to windows	Must
FR-07-03	Staff shall only see controls for their own assigned window	Must
FR-07-04	System shall track each window's current serving ticket, daily transaction count, and avg handling time	Must
FR-07-05	Staff shall be able to mark themselves as on break (window paused, no new calls)	Should

FR-07-06	System shall display window status (open, closed, paused) in supervisor dashboard	Must
----------	---	------

FR-08: Supervisor Monitoring

ID	Requirement	Priority
FR-08-01	Supervisor shall have a live dashboard showing all windows, status, current ticket, and queue lengths	Must
FR-08-02	Supervisor shall be able to see total tickets served, pending, and no-shows for the current day	Must
FR-08-03	Supervisor shall be able to post branch announcements visible on the public display board	Must
FR-08-04	Supervisor shall be able to override ticket states (e.g., re-call a skipped ticket)	Should
FR-08-05	Supervisor shall be able to view per-window daily metrics	Should

FR-09: Notifications

ID	Requirement	Priority
FR-09-01	System shall create an in-app notification when a citizen's booking is confirmed	Must
FR-09-02	System shall create an in-app notification when a citizen's ticket is called	Must
FR-09-03	System shall create an in-app notification when a citizen's ticket is cancelled (by staff or system)	Must
FR-09-04	System shall create a turn-approaching notification when N tickets remain ahead (configurable, default: 3)	Must
FR-09-05	System shall create an in-app notification for branch announcements	Must
FR-09-06	Citizens shall be able to mark notifications as read	Should
FR-09-07	System shall not re-send the same notification type for the same ticket	Must

FR-10: Reports & Analytics

ID	Requirement	Priority
FR-10-01	System shall provide daily summary report: total tickets, served, no-shows, cancellations	Must
FR-10-02	System shall provide average wait time per service per day	Must
FR-10-03	System shall provide peak hours report (tickets per hour, configurable date range)	Must
FR-10-04	System shall provide no-show and cancellation rate metrics	Must
FR-10-05	System shall provide per-window performance metrics (transactions per day, avg handling time)	Should
FR-10-06	Reports shall be filterable by service type and date range	Should

FR-10-07	System shall record daily service metrics snapshots at end of day	Must
----------	---	------

FR-11: Public Display Board

ID	Requirement	Priority
FR-11-01	System shall provide an unauthenticated public display page showing current serving ticket number per window	Must
FR-11-02	Display board shall show the next ticket number in queue	Must
FR-11-03	Display board shall show estimated wait time for a fresh booking	Must
FR-11-04	Display board shall show current branch announcements	Must
FR-11-05	Display board shall auto-refresh at least every 10 seconds	Must
FR-11-06	Display board shall be optimized for large-screen rendering (72pt+ ticket numbers)	Must

FR-12: AI Wait Time Prediction

ID	Requirement	Priority
FR-12-01	System shall compute predicted wait time for any given queue position using heuristic model	Must
FR-12-02	Prediction shall factor in: tickets ahead, avg handling time, active windows, historical throughput	Must
FR-12-03	Prediction shall degrade gracefully if historical data is unavailable	Must
FR-12-04	Prediction shall be exposed via API for both authenticated and public consumers	Must
FR-12-05	System shall periodically store prediction snapshots for accuracy auditing	Could

FR-13: Localization

ID	Requirement	Priority
FR-13-01	UI shall support Arabic (RTL) and English (LTR) languages	Must
FR-13-02	Language preference shall be stored per user and per session for guests	Must
FR-13-03	All system-generated notifications and messages shall be available in both languages	Must
FR-13-04	Service names shall have Arabic and English variants stored separately	Must
FR-13-05	Date/time formatting shall follow locale conventions (Gregorian calendar for MVP)	Should

FR-14: Audit Logging

ID	Requirement	Priority
FR-14-01	System shall create audit log entries for all ticket state transitions	Must

FR-14-02	System shall log all authentication events	Must
FR-14-03	System shall log all role assignments and revocations	Must
FR-14-04	System shall log all service and window configuration changes	Must
FR-14-05	System shall log supervisor override actions	Must
FR-14-06	Audit logs shall be immutable (no update or delete operations)	Must
FR-14-07	Audit logs shall store: actor_id, action, entity_type, entity_id, old_value, new_value, timestamp, ip_address	Must

FR-15: Administration

ID	Requirement	Priority
FR-15-01	Admin shall be able to configure queue rules: grace_period, cancellation_cutoff, max_active_bookings, turn_approaching_threshold	Must
FR-15-02	Admin shall be able to configure branch operating hours	Must
FR-15-03	Admin shall be able to view and search audit logs	Must
FR-15-04	Admin shall be able to reset daily queue counters (for next day)	Must
FR-15-05	Admin shall be able to manage branch announcements	Should

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API response time ≤ 500 ms for 95th percentile under 100 concurrent users
NFR-02	Performance	Public display board refresh ≤ 10 seconds
NFR-03	Availability	System uptime $\geq 99.5\%$ during branch operating hours
NFR-04	Scalability	Architecture supports horizontal scaling post-MVP (stateless API layer)
NFR-05	Security	All API endpoints require HTTPS
NFR-06	Security	JWT tokens must be validated on every protected request
NFR-07	Security	SQL injection and XSS protections must be applied at framework level
NFR-08	Usability	Citizen booking flow completable in ≤ 3 steps on mobile
NFR-09	Accessibility	Color contrast ratio $\geq 4.5:1$; keyboard navigable
NFR-10	Localization	RTL layout fully functional without layout breaks
NFR-11	Reliability	Queue engine operations must be ACID-compliant
NFR-12	Reliability	Duplicate booking prevention via database-level unique constraints
NFR-13	Observability	Application logs (structured JSON) to stdout for PaaS log aggregation
NFR-14	Data Retention	Audit logs retained for minimum 1 year
NFR-15	Browser Support	Chrome 90+, Safari 14+, Firefox 88+, Edge 90+

5. Business Rules

Rule ID	Rule
BR-01	A citizen may hold at most one active booking at any time
BR-02	Bookings are only accepted during branch operating hours
BR-03	A booking may only be cancelled before the cancellation cutoff time
BR-04	A ticket in "called" state has a grace period; after grace period it becomes no_show
BR-05	Queue numbers are unique per service per day and reset at midnight
BR-06	A window may only call a ticket from a service it is assigned to
BR-07	A staff member may only be assigned to one window at a time
BR-08	Daily capacity is counted at booking time; cancellations free the slot
BR-09	A service with active bookings cannot be deleted (only deactivated)
BR-10	Admin and Supervisor roles may not be self-assigned (require existing admin)

6. Missing Requirements & Edge Cases

6.1 Identified Missing Requirements

ID	Missing Requirement	Resolution
MR-01	No definition of queue capacity per time slot or per day	Add daily_capacity and slot_capacity to service config
MR-02	No grace period behavior defined	Add configurable grace period (default: 5 min); after grace → ticket auto-skipped
MR-03	No max active bookings per citizen defined	Add max_active_bookings config (default: 1 per citizen per day)
MR-04	No definition of branch operating hours	Add branch_hours config (open time, close time, days)
MR-05	No cancellation cutoff rule	Add cancellation_cutoff_minutes config (default: 10 min before estimated time)
MR-06	No ticket priority / VIP / disability queue	Stub priority field on ticket; full feature is Phase 2
MR-07	No definition of what happens when all windows close mid-queue	Add branch_close procedure: notify waiting citizens, auto-cancel remaining
MR-08	No check-in mechanism specified	Add manual check-in by citizen (button)
MR-09	No announcement/broadcast capability for branch	Add branch_announcements entity for supervisor-posted messages
MR-10	No definition of how queue numbers are formatted	Define format: {ServiceCode}{DailySequence} e.g., NID-042

6.2 Operational Edge Cases

ID	Edge Case	Handling
----	-----------	----------

EC-01	All service windows close while citizens are waiting	System notifies all waiting tickets; supervisor must reopen or trigger close procedure
EC-02	Citizen's turn arrives but app is closed	Ticket enters "called" state; grace period timer starts; auto-skips to no_show after grace
EC-03	Staff accidentally marks wrong ticket as done	Audit log captures action; supervisor can revert for a configurable window (default: 5 min)
EC-04	Two staff press "Call Next" simultaneously	Optimistic lock on queue engine; only one call succeeds, other returns conflict error
EC-05	Citizen tries to book when daily capacity is reached	Booking rejected with QUEUE_FULL error; citizen sees "No more slots today"
EC-06	Branch announcement posted after citizens already booked	All booked citizens for the day receive in-app notification of announcement
EC-07	Citizen checks in before window opens (very early arrival)	Check-in accepted but status stays checked_in until window opens
EC-08	Network drops for staff during calling operation	Frontend retries with idempotency key; no double-calling
EC-09	Admin deletes a service that has active bookings	Deletion blocked; must first close bookings or mark service inactive
EC-10	Two citizens book simultaneously for last slot	Optimistic locking; one succeeds, other receives SLOT_UNAVAILABLE

7. External Interface Requirements

Interface	Type	Description
Citizen Web App	Browser (React SPA)	Mobile-first responsive UI
Staff Web App	Browser (React SPA)	Desktop-optimized calling interface
Supervisor Dashboard	Browser (React SPA)	Live monitoring + reports
Admin Panel	Browser (React SPA)	Configuration + user management
Public Display Board	Browser (React SPA)	Large-screen unauthenticated view
REST API	JSON over HTTPS	All client-server communication
SSE Endpoint	Server-Sent Events	Real-time queue updates
PostgreSQL	TCP/IP	Primary data store
Redis	TCP/IP	Session cache + queue state (optional MVP)